

Institut Supérieure d'Informatique et de
Multimédia de Sfax



Atelier de Programmation

Chapitre 3

Les structures de contrôle de flux

Enseignant : Ramzi ZOUARI

1^{ère} année License en Sciences de l'Informatique

Année universitaire : 2025-2026

Plan

1. Introduction
2. Les instructions alternatives
3. Les instructions répétitives
4. Les instructions de branchement

1. Introduction

- Le langage C met à disposition des structures de contrôle de flux par lesquelles on peut piloter le déroulement d'un programme
- Ces instructions sont parfois nécessaires pour répéter certaines phases de traitement ou encore rendre l'exécution de certaines instructions dépendantes d'une certaine condition
- Nous distinguerons trois groupes de structures de contrôle de flux:
 - Les instructions alternatives (tests)
 - Les instructions répétitives (itérations)
 - Les instructions de branchement

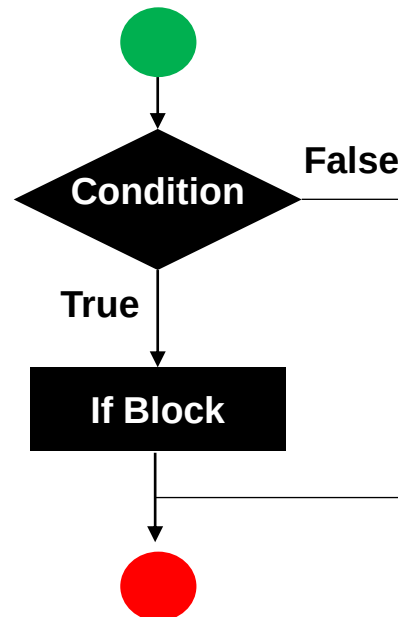
2. Les instructions alternatives

- Les structures alternatives permettent d'exécuter ou de ne pas exécuter certaines instructions du programme, en respectant des règles définies par le programmeur
- En C, les instructions alternatives sont :
 - **if** : test simple
 - **if – else** : test avec alternative
 - **switch** : test multiple

2.1. Test if

- Les structures alternatives permettent d'exécuter ou de ne pas exécuter certaines instructions du programme, en respectant des règles définies par le programmeur
- L'instruction **if** permet de ne faire exécuter certaines instructions que si une certaine condition est évaluée comme **TRUE**

Syntaxe
<pre>if (<expression>) { <instruction1>; <instruction2>; <instructionN>; }</pre>



Exemple 1
<pre>int a=3, b=2; if (a > b) { printf("a est plus grand que b"); } printf ("fin du programme");</pre>
Affichage 1
a est plus grand que b fin du programme

Exemple 2
<pre>int a=2, b=3; if (a > b) { printf("a est plus grand que b"); } printf ("fin du programme");</pre>
Affichage 2
fin du programme

2.1. Test if

- Les conditions relatives à une instruction **if** représentent généralement des expressions de comparaison utilisant les symboles : > , >=, <, <=, ==, !=, !
- **Remarque:** Ne pas confondre entre l'opérateur d'affectation (=) et de comparaison (==)

Exemple 1
<pre>int a=3, b=2; if (a == b) { printf("a est égale à b"); } printf ("fin du programme");</pre>
Affichage 1
fin du programme

Exemple 2
<pre>int a=2, b=3; if (a != b) { printf("a est different de b"); } printf ("fin du programme");</pre>
Affichage 2
a est different de b fin du programme

Exemple 3
<pre>int a=0; if (!a) { // a==0 printf("a est égale à 0"); } printf ("fin du programme");</pre>
Affichage 3
a est égale à 0 fin du programme

Exemple 4
<pre>int a=2; if (a) { // a!=0 printf("a est différent de 0"); } printf ("fin du programme");</pre>
Affichage 4
a est différent de 0 fin du programme

2.1. Test if

- L'instruction **if** peut comporter plusieurs conditions. Dans ce cas, les conditions peuvent être liées à travers les trois opérateurs logiques : **&&** **||** **^**

Exemple 1

```
int a=3, b=2;
if (a > b && b>0) {
    printf("a est strictement positif");
}
printf ("fin du programme");
```

Affichage 1

a est plus grand que b
fin du programme

Exemple 2

```
int a=2, b=3;
if (a < b || b>0) {
    printf("b est strictement positif");
}
printf ("fin du programme");
```

Affichage 2

a est strictement positif
fin du programme

Exemple 3

```
int a=2, b=3;
if (a > b ^ b<0) {
    printf("la condition vaut TRUE");
}
printf ("fin du programme");
```

Affichage 3

la condition vaut TRUE
fin du programme

2.1. Test if

- L'instruction **if** peut comporter des conditions avec des opérateurs logiques différentes
- **Remarque:** il est fortement recommandé d'utiliser les parenthèses pour prioriser des opérateurs logiques par rapport à d'autres

Exemple 1

```
int a=3, b=2;
if ( (a<b && b>0) || a!=b ) {
    printf("La condition vaut TRUE");
}
printf ("fin du programme");
```

Affichage 1

```
La condition vaut TRUE
fin du programme
```

Exemple 2

```
int a=2, b=3;
if ( a<b && (b>0 || a!=b) ) {
    printf("La condition vaut TRUE");
}
printf ("fin du programme");
```

Affichage 2

```
fin du programme
```

Exemple 3

```
int a=2, b=3;
if ( (a<b && b>0) ^ (a==b || b==3) ) {
    printf("La condition vaut TRUE");
}
printf ("fin du programme");
```

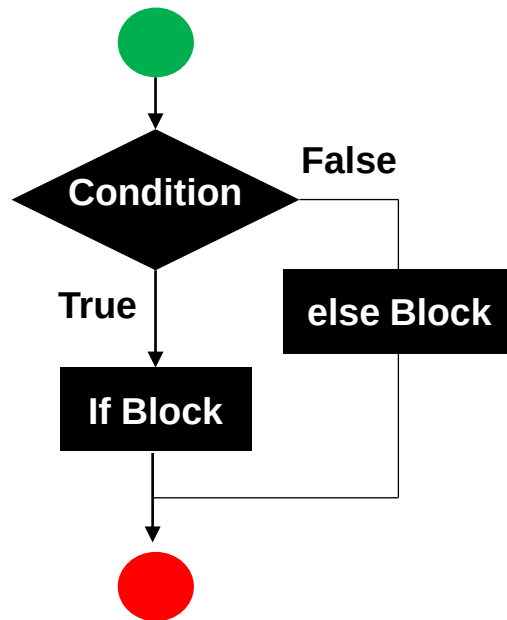
Affichage 3

```
la condition vaut TRUE
fin du programme
```

2.2. Test avec alternative (if – else)

- La structure avec alternative **if–else** permet d'exécuter les instructions de **if** si la condition est remplie, sinon exécute les instructions du mot-clé **else**

Syntaxe
<pre>if (<expression>) { <instruction1>; <instructionN>; } else { <instruction1>; ... <instructionN>; }</pre>



Exemple 1
<pre>int a=3; if (a > 0) { printf("a est positif"); } else { printf ("b est negatif"); }</pre>
Affichage 1
a est positif

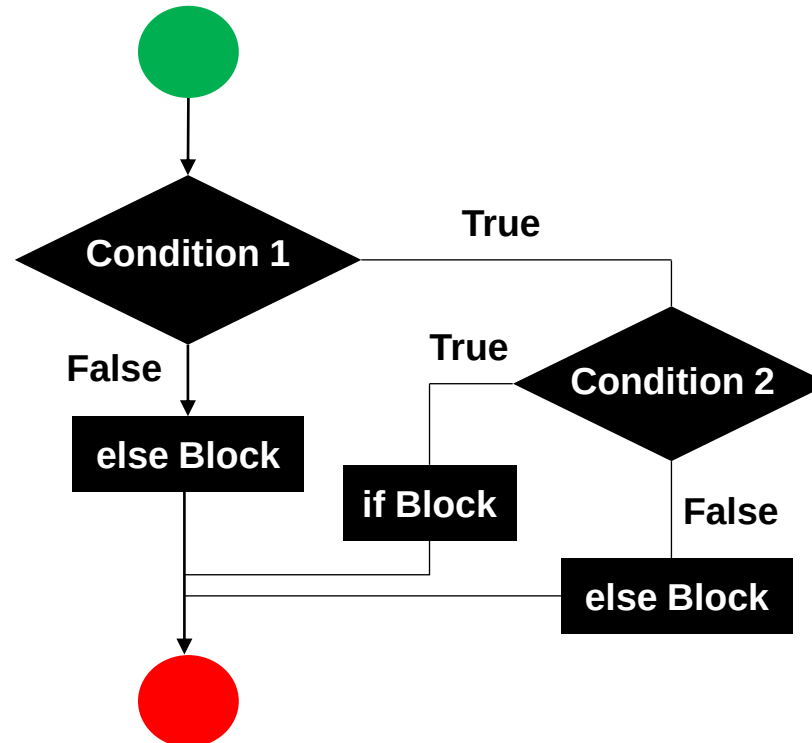
Exemple 2
<pre>int a=-3; if (a > 0) { printf("a est positif"); } else { printf ("b est negatif"); }</pre>
Affichage 2
b est negatif

2.3. Tests imbriqués

- L'instruction **if-else** peut contenir une autre instruction **if-else** dans la branche **if**, ou la branche **else**, ou les deux simultanément

Syntaxe 1

```
if ( <expression1> ) {  
    if ( <expression2> ) {  
        <instruction1>;  
    }  
    else {  
        <instruction2>;  
    }  
}  
else {  
    <instruction3>;  
}
```



Exemple 1

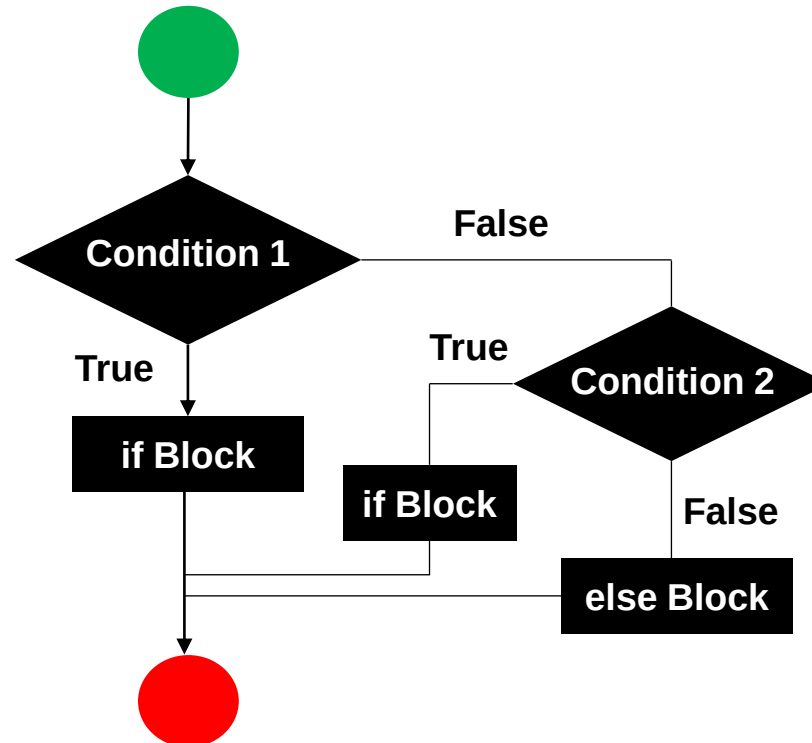
```
int a=8;  
if ( a >= 0 ) {  
    if ( a ==0 ) {  
        printf("a est nulle");  
    }  
    else {  
        printf("a est positive");  
    }  
}  
else {  
    printf ("a est negative");  
}
```

2.3. Tests imbriqués

- L'instruction **if-else** peut contenir une autre instruction **if-else** dans la branche **if**, ou la branche **else**, ou les deux simultanément

Syntaxe 2

```
if ( <expression1> ) {  
    <instruction1>;  
}  
else {  
    if (<expression2>) {  
        <instruction2>;  
    }  
    else {  
        <instruction3>;  
    }  
}
```



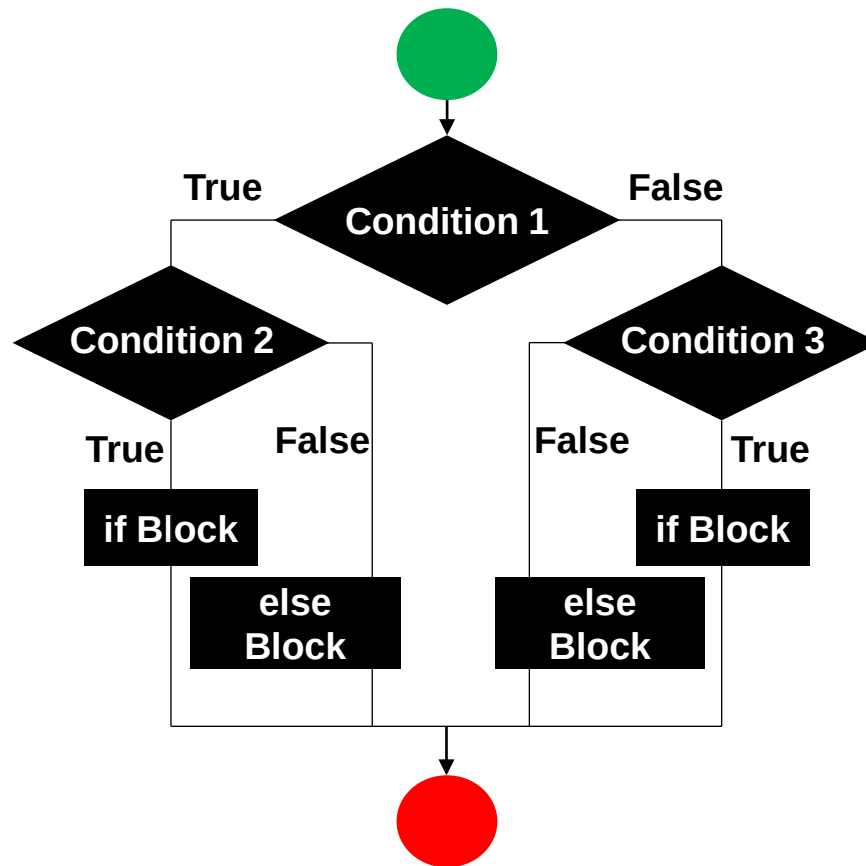
Exemple 2

```
int a = -8;  
if ( a < 0 ) {  
    printf("a est negative");  
}  
else {  
    if ( a == 0 ) {  
        printf("a est nulle");  
    }  
    else {  
        printf ("a est positive");  
    }  
}
```

2.3. Tests imbriqués

Syntaxe 3

```
if ( <expression1> ) {  
    if (<expression2>) {  
        <instruction1>;  
    }  
    else {  
        <instruction2>;  
    }  
}  
else {  
    if (<expression2>) {  
        <instruction3>;  
    }  
    else {  
        <instruction4>;  
    }  
}
```



Exemple 3

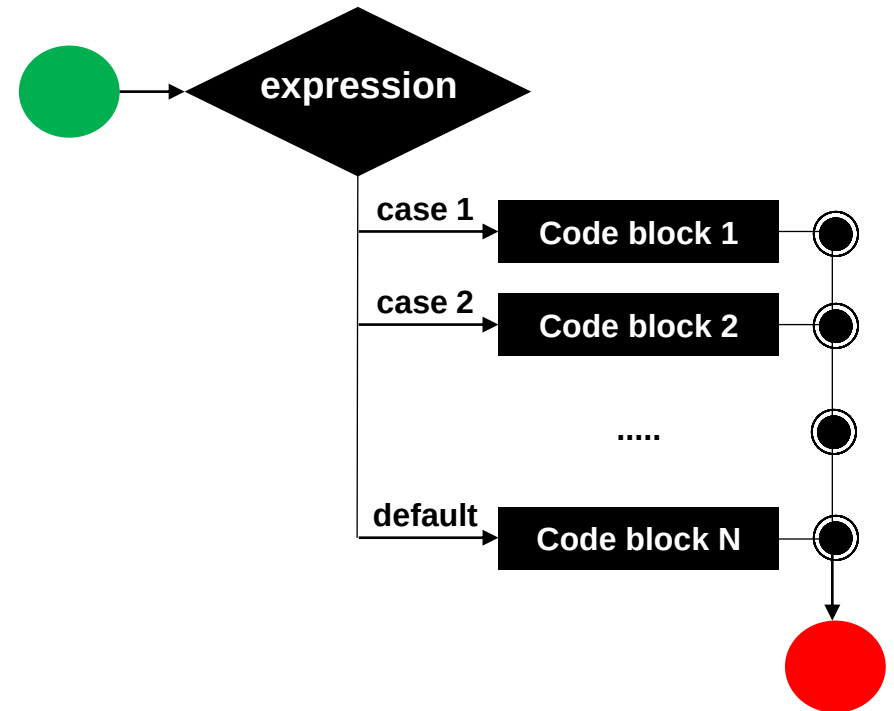
```
Int a=4, b= -2, c;  
if ( a > 0 ) {  
    if ( b > 0 ) {  
        c = a * b;  
    }  
    else {  
        c = a * -b;  
    }  
}  
else {  
    if (b > 0) {  
        c = -a * b ;  
    }  
    else {  
        c = -a * -b;  
    }  
}
```

2.4. Test multiple switch

- L'instruction **switch** permet de tester l'égalité d'une expression par rapport à une liste de valeurs (**case**), contrairement à l'instruction **if-else** qui renferme seulement deux choix

Syntaxe

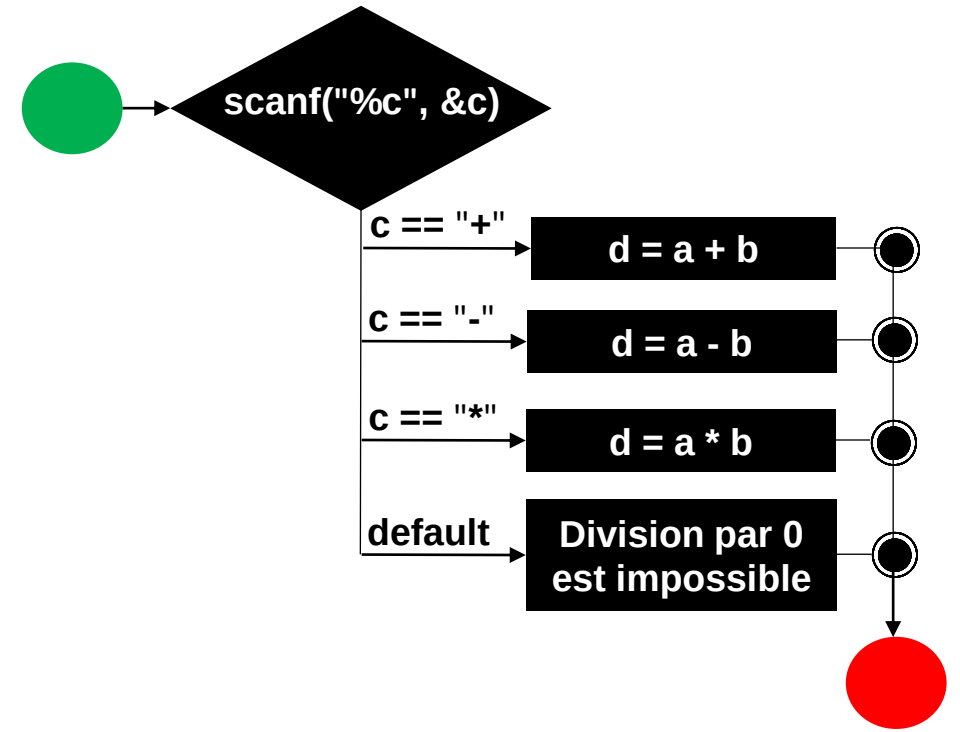
```
switch (<expression>) :  
{  
  case value_1: // si expression retourne value_1  
    <instructions> ;  
    break ; // sortir de la structure switch  
  
  ... ..  
  
  case value_N:  
    <instructions> ;  
    break ;  
  
  default : // instructions à exécuter si aucune valeur est vrai  
    <instructions> ;  
    break ;  
}
```



2.4. Test multiple switch

Exemple

```
int a = 2 , b = 0; d;  
char c;  
printf ("Donner un opérateur arithmetique") ;  
scanf ("%c", &c);  
switch (c) :  
{  
    case "+":  
        d = a + b;  
        break ;  
    case "-":  
        d = a - b;  
        break ;  
    case "*":  
        d = a * b;  
        break ;  
    default :  
        printf ("Division par 0 est impossible") ;  
        break ;  
}
```



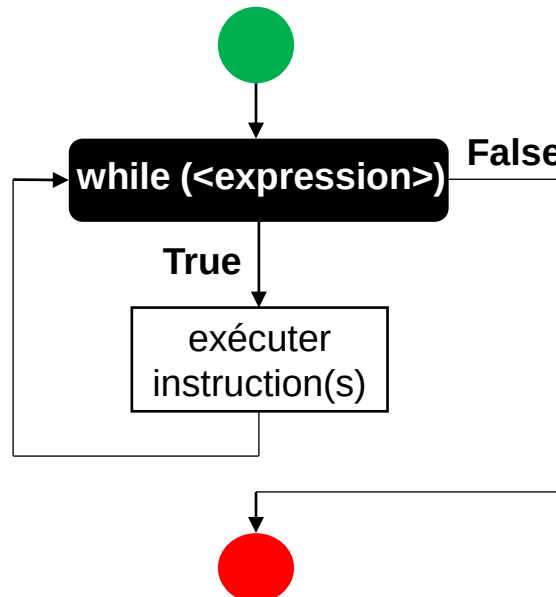
3. Les instructions répétitives

- Les structures répétitives permettent d'exécuter plusieurs fois certaines instructions du programme sans avoir besoin de les réécrire à chaque itération
- Le langage C dispose de trois structures répétitives:
 - **while**
 - **for**
 - **do ... while**
- Les structures (boucles) **while** et **for** procèdent à la vérification de la condition avant que les instructions dépendantes de la condition soient exécutées
- La boucle **do...while** commence par exécuter les instructions dépendantes de la condition, puis vérifie à la fin de la boucle si la condition permet une reprise des instructions

3.1. La boucle while

- La boucle **while** permet de faire répéter l'exécution d'instructions tant qu'une certaine condition est remplie c.à.d. évaluée à TRUE

Syntaxe while
while (<expression>) <instruction1>;
while (<expression>) { <instruction1>; <instruction2>; <instructionN>; }



Exemple
<pre>int a = 10, b=1 ; while (a !=b) { if (a/b == 0) { printf("%d est un diviseur de %d\n", b, a); b +=1; } }</pre>
Affichage
<pre>2 est un diviseur de 10 5 est un diviseur de 10</pre>

3.1. La boucle while

Enoncé

Ecrire en C un programme qui permet de calculer le plus grand diviseur commun PGCD entre deux entiers a et b

Exemple

a = 48, b = 18

48 > 18 → a = a – b = 48 – 18 = 30

30 > 18 → a = a – b = 30 – 18 = 12

12 < 18 → b = b – a = 18 – 12 = 6

12 > 6 → a = a – b = 12 – 6 = 6

6 = 6 → a = b → fin programme (PGCD = 6)

Implémentation C

```
#include<stdio.h>
int main () {
    printf("Donner a:");
    scanf("%d", &a);
    printf("Donner b:");
    scanf("%d", &b);
    while ( a != b) {
        if (a > b)
            a = a – b;
        else
            b = b – a;
    }
    printf("PGCD=%d", a);
    return 0;
}
```

3.2. La boucle for

- La boucle **for** permet d'exécuter (répéter) des instructions tant que la condition définie en amont est remplie c.à.d. évaluée à TRUE
- La syntaxe de la boucle **for** renferme:
 - Une initialisation des variables de contrôle <expression_I>
 - Une condition de bouclage <expression_C> : Elle doit être évaluée à True pour exécuter les instructions
 - Une réinitialisation des variables de contrôle <expression_R>

Syntaxe for

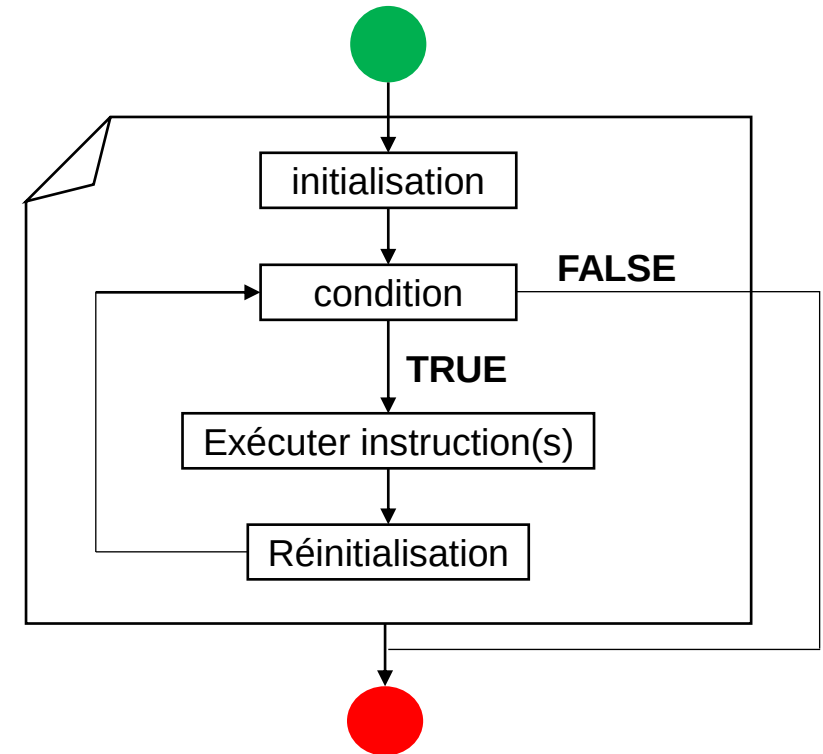
```
for (<expression_I>; <expression_C>; <expression_R> )  
    <instruction1>;
```

Syntaxe for

```
for (<expression_I>; <expression_C>; <expression_R> )  
{  
    <instruction1>;  
    ....  
    <instructionN>;  
}
```

3.2. La boucle for

- **Initialisation:** exécutée seulement à l'entrée de la boucle
- **Condition:** exécutée à chaque itération
 - **TRUE** : exécuter le(s) instruction(s) dépendantes
 - **FALSE**: sortir de la boucle et continuer le programme
- **Réinitialisation:** exécutée après l'exécution de(s) instruction(s) de l'itération



3.2. La boucle for

- <expression_I> est exécutée seulement à la première itération de la boucle **for** permet
- <expression_C> est exécutée à chaque itération de la boucle for
 - Si <expression_C> est TRUE, les instructions qui en dépendent seront exécutées
 - Si <expression_C> est FAUSE, fin de la boucle for
- <expression_R> est exécuté à la fin de chaque itération c.à.d. après la fin de l'exécution des instructions de la boucle for

Exemple

```
int i;  
  
for ( i=0; i<10 ; i++ )  
    printf("chiffre %d", i);
```

Exécution

```
itération 1: i=0, 0 < 10 est vrai , "chiffre 0", i=0+1  
itération 2: i=1, 1 < 10 est vrai , "chiffre 1", i=1+1  
.....  
itération 10: i=10, 10<10 est Faux (sortir de la boucle for)
```

3.2. La boucle for

Exemple

```
int i;  
for ( i=10; i > 1 ; i -- )  
    printf("chiffre %d", i);
```

Exécution

Itération 1 : i=10, 10 > 1 est vrai , "chiffre 10", i = 10 - 1
Itération 2 : i=9, 9 > 1 est vrai , "chiffre 9", i = 9 - 1
... ..
Itération 10 : i=1, 1 > 1 est **Faux** (sortir de la boucle for)

Exemple

```
int i;  
for ( i = 0; i < 10 ; i = i + 3 )  
    printf("chiffre %d", i);
```

Exécution

Itération 1 : i=0, 0 < 10 est vrai , "chiffre 0", i=0+3
Itération 2 : i=3, 3 < 10 est vrai , "chiffre 3", i = 3 + 3
Itération 3 : i=6, 6 < 10 est vrai , "chiffre 6", i = 6 + 3
Itération 4 : i=9, 9 < 10 est vrai , "chiffre 9", i = 9 + 3
Itération 5 : i=12, 12 < 10 est **Faux** → sortir de la boucle for

3.2. La boucle for

Enoncé

Ecrire en C un programme qui permet de retourner les nombre impairs entre 0 et 10

Exemple

$i=0 \rightarrow 0\%2 = 0 \rightarrow$ (passer à l'itération suivante)

$i=1 \rightarrow 1\%2 \neq 0 \rightarrow$ 1 est impair

$i=2 \rightarrow 2\%2 = 0 \rightarrow$ (passer à l'itération suivante)

... ..

$i=9 \rightarrow 9\%2 \neq 0 \rightarrow$ 9 est impair

$i=10 \rightarrow 10\%10 = 0 \rightarrow$ sortir de la boucle for

Implémentation C

```
#include<stdio.h>
int main () {
    for ( i = 0 ; i<=10 ; i++) {
        if (i % 2)
            printf("%d est impair", a);
    }
    return 0;
}
```

3.2. La boucle for

Enoncé

Ecrire en C un programme qui permet de retourner les nombres premiers entre 2 et 10

Exemple

i = 2

j=2 → j n'est pas ≤ i/2 (sortir de la boucle for j)

nb_div = 0 → 2 est un nombre premier

i = 3

j=2 → j n'est pas ≤ i/2 (sortir de la boucle for j)

nb_div = 0 → 3 est un nombre premier

i = 4

j=2 → j ≤ i/2 → i%j ==0 → nb_div = 1

j=3 → j'est pas ≤ i/2 → sortir de la boucle for j

Implémentation C

```
#include<stdio.h>
int main () {
    int i, j, nb_div;
    for ( i = 2 ; i <= 10 ; i+1) {
        nb_div = 0 ; // compter le nombre de diviseurs de i
        for (j = 2 ; j <= i/2 ; j+1) {
            if (i % j == 0) // i est divisible par j
                nb_div += 1;
        }
        if (nb_div == 0)
            printf("%d est un nombre premier", i);
    }
    return 0;
}
```

3.2. La boucle for

Enoncé

Ecrire en C un programme qui permet de trouver le plus petit multiplieur commun entre 8 et 12

Exemple

i = 23

$((23 \% 12) == 0 \ \&\& \ (23 \% 8) == 0)$ est **Faux** → **i + 1**

i = 24

$(24 \% 12) == 0 \ \&\& \ (24 \% 8) == 0$ est **Vrai** → **break**

Implémentation C

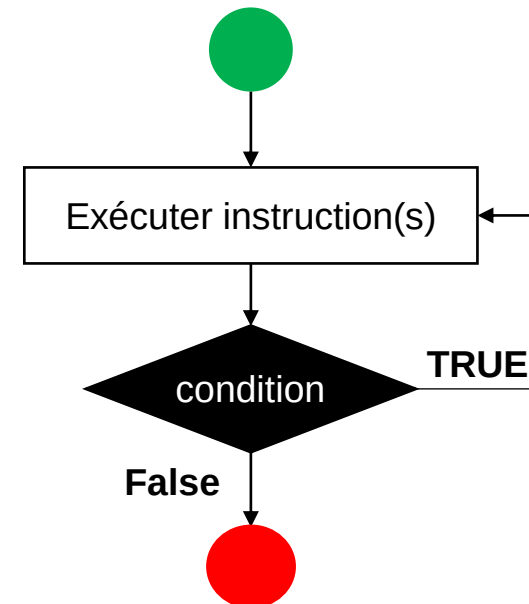
```
#include<stdio.h>
int main () {
    int i;
    for ( i = 1 ; i <= 100 ; i+1) {
        if ((i % 12 == 0) && (i%8 ==0))
            break ; // sortie prématurée de la boucle for
    }
    printf("Le PPCM de 8 et 12 est %d", i);
    return 0;
}
```

3.3. La boucle do ... while

- Contrairement aux structures **for** et **while**, la boucle **do...while** exécute le(s) instruction(s) avant de tester la condition (les instructions sont exécutées au moins une fois)
 - Si <condition> vaut **TRUE**, les instructions sont exécutées de nouveau,
 - Si <condition> vaut **FALSE**, sortir de la boucle **do ... while**

Syntaxe
do <instruction>; while (<condition>;

Syntaxe
do { <instruction1>; <instructionN>; } while (<condition>;



3.3. La boucle do ... while

Exemple

```
int a=2, b = 3;  
do  
    a = a + b;  
while (a <= 7)
```

Exécution

Itération 1

- $a = 2+3 = 5 \rightarrow 5 \leq 7$ est Vrai

Itération 2

- $a = 5+3 = 8 \rightarrow 8 \leq 7$ est **Faux** (sortir de la boucle)

Exemple

```
int a=2, b = 3;  
do {  
    a = a + b;  
    b = b + 3;  
} while (a <= 10)
```

Exécution

Itération 1

- $a = 2+3 = 5, b=3+3=6 \rightarrow 5 \leq 10$ est Vrai

Itération 2

- $a=5+6=11, b=6+3=9 \rightarrow 11 \leq 10$ est **Faux** (sortir de la boucle)

3.3. La boucle do ... while

Enoncé

Ecrire en C un programme qui permet de calculer le PGCD de a et b en utilisant la boucle **do...while**

Exemple

a=100, b= 75

itération 1: a>b → a=25, b=75 → a!=b est Vrai

itération 2: a<b → a=25, b=50 → a!=b est Vrai

itération 3: a<b → a=25, b=25 → a!=b est **Faux**

Implémentation C

```
#include<stdio.h>
int main () {
    int a=100, b=25;
    do {
        if (a > b)
            a = a - b ;
        else
            b = b - a;
    } while (a!=b);
    printf("PGCD est %d", a);
    return 0;
}
```

4. Les instructions de branchement

- Elle permettent de transférer le contrôle d'un programme d'une instruction à une autre c.à.d. les instructions d'un programme peuvent ne pas s'exécuter dans le même ordre de leurs apparitions
- Les instructions de branchement sont:
 - break
 - continue
 - goto
 - return (sera traitée dans le chapitre 5)

4.1 instruction de branchement break

- L'instruction **break** ne peut s'utiliser qu'à l'intérieur des structures **for**, **while**, **do..while**, et **switch**. Elle provoque l'interruption immédiate de ces instructions
- Le programme continue son exécution à partir de l'instruction qui suit la structure interrompue

Exemple 1

```
#include<stdio.h>
int main() {
    int i, p=1;
    for (i=1; i<10; i++) {
        if (p==6)
            break;
        else
            p = p*i;
    }
    printf("p=%d", p);
    return 0;
}
```

Exécution

```
i = 1 → p=1*1=1
i = 2 → p=1*2=2
i = 3 → p=2*3=6
i = 4 → interruption break (sortir de la boucle for)
Affichage: p=6
```

4.1 instruction de branchement break

Exemple 2

```
#include<stdio.h>
int main() {
    int a, nb = 150, essai=1;
    while (1) { // boucle infini
        printf("Deviner la valeur de nb");
        scanf("%d", &a)
        if (nb == a)
            break;
        else {
            if (essai ==3)
                break;
            else
                essai += 1;
        }
    }
    if (nb_essai == 3)
        printf("Nombre maximal de tentatives est atteint");
    else
        printf("Bravo !!! ");
    return 0;
}
```

Example 3

```
#include<stdio.h>
int main() {
    int a, nb = 150, essai=0;
    do {
        printf("Deviner la valeur de nb");
        scanf("%d", &a);
        essai +=1;
        if ( (essai == 3) && (nb !=a))
            break;
    } while (nb != a);
    if (nb != a)
        printf("Nombre maximal de tentatives est atteint");
    else
        printf("Bravo !!! ");
    return 0;
}
```

4.1 instruction de branchement break

- Dans le cas des boucles imbriquées, l'instruction **break** ne mettra fin qu'à la structure intérieur dans laquelle elle est appelée

Example

```
#include<stdio.h>
int main() {
    int i, j, c=60;
    for (i=1; i<=4; i++) {
        for (j=1; j<=40; j++)
            if ( (i*j) == c) {
                printf("%d * %d = %d", i, j, c);
                break;
            }
    }
    return 0;
}
```

Exécution

i=1 (rien à afficher)
i=2 → 2*30=60
i=3 → 3*20=60
i=4 → 4*15=60

4.1 instruction de branchement continue

- L'instruction **continue** permet de sauter un seul passage (itération) dans la boucle.
L'exécution reprend dans l'itération suivante de la boucle

Example

```
#include<stdio.h>
int main() {
    int i;
    for (i=0; i<=4; i++) {
        if (i==0)
            continue;
        else
            printf("\ni=%d", i);
    }
    return 0;
}
```

Exécution

```
i=1
i=2
i=3
i=4
```

4.2 instruction de branchement continue

- L'instruction **continue** permet de sauter un seul passage (itération) dans la boucle.
L'exécution reprend dans l'itération suivante de la boucle

Exemple

```
#include<stdio.h>
int main() {
    int i=1;
    while (i<10) {
        if (i%2) {
            continue;
            i += 1;
        }
        else {
            printf("\ni=%d est pair", i);
            i += 1;
        }
    }
    return 0;
}
```

Exécution

2 est un nombre pair
4 est un nombre pair
6 est un nombre pair
8 est un nombre pair

4.3 instruction de branchement goto

- L'instruction **goto** permet de sauter à un endroit du programme repéré par une étiquette (label). Le programme continue alors à l'instruction qui se trouve à cet endroit-là.

Exemple

```
#include<stdio.h>
int main() {
    int a=15, b;
    saisir : // etiquette
        printf("\nDonner b: ");
        scanf("%d", &b);
    if (a%b)
        goto saisir ; // branchement vers saisir
    else
        printf("%d est divisible par %d", a, b);
    return 0;
}
```

Exécution 1

b = 3
Le programme affiche : 15 est divisible par 3

Exécution 2

b = 6
Le programme affiche : Donner b